

Spring Boot Microservices & Spring Cloud

Advanced Guide — Volume 2

New & Advanced Topics Beyond the Fundamentals

gRPC • Service Mesh (Istio) • Kubernetes Deployment • Prometheus & Grafana

Axon Framework • Event Sourcing • Multi-Tenancy • GraphQL Gateway

Advanced Kafka Patterns • Outbox Pattern • Blue-Green Deployments

Definitions • Architecture • Production-Grade Code Examples

Table of Contents

1. gRPC in Spring Boot Microservices

- 1.1 What is gRPC & Why Use It?
- 1.2 Protocol Buffers (.proto) Definition
- 1.3 gRPC Server – Spring Boot
- 1.4 gRPC Client – Service-to-Service
- 1.5 gRPC Streaming
- 1.6 gRPC vs REST Comparison

2. Service Mesh – Istio with Spring Boot

- 2.1 What is a Service Mesh?
- 2.2 Istio Architecture (Control Plane / Data Plane)
- 2.3 Traffic Management – VirtualService & DestinationRule
- 2.4 mTLS – Automatic Mutual TLS
- 2.5 Canary Deployments with Istio
- 2.6 Observability with Kiali

3. Kubernetes Deployment for Microservices

- 3.1 Kubernetes Core Concepts
- 3.2 Deployment & Service YAML
- 3.3 ConfigMaps & Secrets
- 3.4 Spring Cloud Kubernetes
- 3.5 Horizontal Pod Autoscaler
- 3.6 Liveness & Readiness Probes
- 3.7 Helm Charts for Spring Boot

4. Observability – Prometheus, Grafana & AlertManager

- 4.1 Observability Pillars (Metrics, Logs, Traces)
- 4.2 Micrometer + Prometheus
- 4.3 Grafana Dashboard Setup
- 4.4 Custom Metrics with @Timed & MeterRegistry
- 4.5 AlertManager Rules
- 4.6 Structured Logging with Logback + ELK

5. Advanced Kafka Patterns

- 5.1 Kafka Streams – Real-Time Processing
- 5.2 Transactional Producers
- 5.3 Dead Letter Queue (DLQ)
- 5.4 Schema Registry + Avro
- 5.5 Exactly-Once Semantics
- 5.6 Kafka Connect

6. Transactional Outbox Pattern

- 6.1 The Dual-Write Problem
- 6.2 Outbox Pattern Architecture
- 6.3 Implementation with Spring + CDC
- 6.4 Debezium Integration

7. Event Sourcing & Axon Framework

- 7.1 Event Sourcing Concepts
- 7.2 Axon Framework Architecture
- 7.3 Aggregate, Command & Event
- 7.4 Event Store
- 7.5 Projections & Query Models
- 7.6 Sagas in Axon

8. GraphQL API Gateway

- 8.1 Why GraphQL at the Gateway?
- 8.2 Spring GraphQL Setup
- 8.3 Schema Federation
- 8.4 DataLoader & N+1 Prevention
- 8.5 Subscriptions (WebSocket)

9. Multi-Tenancy in Microservices

- 9.1 Multi-Tenancy Strategies
- 9.2 Tenant Isolation per Database
- 9.3 Tenant Context Propagation
- 9.4 Row-Level Security

10. Advanced Deployment Patterns

- 10.1 Blue-Green Deployment
- 10.2 Canary Release
- 10.3 Feature Flags with Spring
- 10.4 Zero-Downtime Database Migrations
- 10.5 Rate Limiting at Service Level

CHAPTER 1

gRPC in Spring Boot Microservices

High-Performance Binary RPC for Service-to-Service Communication

1.1 What is gRPC & Why Use It?

gRPC (Google Remote Procedure Call)

gRPC is a high-performance, open-source RPC framework developed by Google. It uses Protocol Buffers (Protobuf) as its interface definition language and serialisation format, and HTTP/2 as the transport protocol. gRPC supports unary, server streaming, client streaming, and bidirectional streaming communication patterns.

Feature	REST / JSON	gRPC / Protobuf
Protocol	HTTP/1.1	HTTP/2 (multiplexing, header compression)
Data Format	JSON (text, human-readable)	Protobuf (binary, ~5-10x smaller)
Contract	OpenAPI/Swagger (optional)	Strongly-typed .proto file (required)
Streaming	Limited (SSE / WebSocket workaround)	Native: unary, server, client, bidi
Code Generation	Manual or OpenAPI codegen	Automatic from .proto for all languages
Browser Support	Full native support	Requires gRPC-Web proxy
Performance	Moderate	Significantly faster for internal calls
Best For	Public APIs, browser clients	Internal microservice communication

1.2 Protocol Buffers – .proto Definition

Protocol Buffers (Protobuf)

Protobuf is Google's language-neutral, platform-neutral, extensible mechanism for serialising structured data. You define your data structures and service methods in a .proto file. The protoc compiler generates strongly-typed client and server code in your target language (Java, Go, Python, etc.).

`product_service.proto` – Service definition

```
// src/main/proto/product_service.proto
syntax = "proto3";

package com.example.grpc;

option java_multiple_files = true;
option java_package = "com.example.grpc.proto";
option java_outer_classname = "ProductServiceProto";
```



```

<dependency>
<groupId>com.google.protobuf</groupId>
<artifactId>protobuf-java</artifactId>
<version>3.25.0</version>
</dependency>

<!-- Build plugin to compile .proto files -->
<plugin>
<groupId>org.xolstice.maven.plugins</groupId>
<artifactId>protobuf-maven-plugin</artifactId>
<version>0.6.1</version>
<configuration>
<protocArtifact>com.google.protobuf:protoc:3.25.0:exe:${os.detected.classifier}</protocArtifact>
<pluginId>grpc-java</pluginId>
<pluginArtifact>io.grpc:protoc-gen-grpc-java:1.60.0:exe:${os.detected.classifier}</pluginArtifact>
</configuration>
</plugin>

```

1.3 gRPC Server Implementation

gRPC Server Implementation

```

// gRPC Server - Product Service
// Extends the generated base class from .proto compilation
@GrpcService
public class ProductGrpcServiceImpl
extends ProductGrpcServiceGrpc.ProductGrpcServiceImplBase {

    private final ProductRepository productRepo;

    // Unary RPC - GetProduct
    @Override
    public void getProduct(ProductRequest request,
        StreamObserver<ProductResponse> responseObserver) {
        try {
            Product product = productRepo
                .findById(request.getProductId())
                .orElseThrow(() -> new StatusRuntimeException(
                    Status.NOT_FOUND.withDescription("Product not found: " + request.getProductId())));

            ProductResponse response = ProductResponse.newBuilder()
                .setId(product.getId())
                .setName(product.getName())
                .setPrice(product.getPrice())
                .setStock(product.getStock())
                .setCategory(product.getCategory())
                .build();

            responseObserver.onNext(response);

```

```

responseObserver.onCompleted();
} catch (StatusRuntimeException e) {
responseObserver.onError(e);
}
}

// Server Streaming RPC - ListProducts
@Override
public void listProducts(Empty request,
StreamObserver<ProductResponse> responseObserver) {
productRepo.findAll().forEach(product -> {
responseObserver.onNext(toResponse(product));
});
responseObserver.onCompleted();
}

private ProductResponse toResponse(Product p) {
return ProductResponse.newBuilder()
.setId(p.getId()).setName(p.getName())
.setPrice(p.getPrice()).setStock(p.getStock())
.build();
}
}

# application.yml - gRPC server port
grpc:
server:
port: 9090

```

1.4 gRPC Client in Order Service

gRPC Client with Blocking and Async stubs

```

// gRPC Client - Order Service calling Product Service
@Service
public class ProductGrpcClient {

// @GrpcClient auto-creates the channel + stub from config
@GrpcClient("product-service")
private ProductGrpcServiceGrpc.ProductGrpcServiceBlockingStub productStub;

public ProductResponse getProduct(Long productId) {
ProductRequest request = ProductRequest.newBuilder()
.setProductId(productId)
.build();
try {
return productStub.getProduct(request);
} catch (StatusRuntimeException e) {
if (e.getStatus().getCode() == Status.Code.NOT_FOUND) {
throw new ResourceNotFoundException("Product not found: " + productId);
}
}
}
}

```

```

    }
    throw new ServiceUnavailableException("Product service error", e);
    }
}

// Reactive (non-blocking) using async stub
@GrpcClient("product-service")
private ProductGrpcServiceGrpc.ProductGrpcServiceStub asyncProductStub;

public Flux<ProductResponse> streamAllProducts() {
    return Flux.create(sink -> {
        asyncProductStub.listProducts(Empty.getDefaultInstance(),
            new StreamObserver<ProductResponse>() {
                public void onNext(ProductResponse r) { sink.next(r); }
                public void onError(Throwable t) { sink.error(t); }
                public void onCompleted() { sink.complete(); }
            });
    });
}

# order-service application.yml
grpc:
  client:
    product-service:
      address: "discovery:///product-service" # Eureka discovery
      negotiation-type: plaintext

```


CHAPTER 2

Service Mesh – Istio with Spring Boot

Traffic Management, mTLS, Observability & Canary Deployments

2.1 What is a Service Mesh?

Service Mesh

A service mesh is a dedicated infrastructure layer that handles service-to-service communication. It provides traffic management, security (mTLS), observability, and reliability features WITHOUT requiring any code changes in your microservices. Each microservice pod gets a sidecar proxy (Envoy) injected automatically, and all traffic flows through these proxies.

Service Mesh Feature	Without Mesh (Spring Cloud)	With Istio
Load Balancing	Spring Cloud LoadBalancer in app	Envoy sidecar proxy (transparent)
Circuit Breaking	Resilience4j in app code	Istio DestinationRule (infrastructure)
Retry / Timeout	Resilience4j config in each service	VirtualService YAML (zero code)
mTLS	Manual cert management in each app	Automatic mutual TLS (cert rotation)
Tracing	Micrometer + instrumentation code	Automatic trace injection in Envoy
Traffic Splitting	Custom code / multiple deployments	VirtualService weight-based routing
Metrics	Micrometer endpoint per service	Unified Prometheus scraping via Envoy

2.2 Istio Traffic Management

Istio VirtualService + DestinationRule for Canary

```
# Canary Deployment: route 90% to v1, 10% to v2 of product-service
---
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: product-service-vs
spec:
  hosts:
  - product-service
  http:
  - match:
  - headers:
    x-canary-user:
      exact: "true" # header-based routing for testers
```

```

route:
- destination:
host: product-service
subset: v2
weight: 100
- route: # default traffic split
- destination:
host: product-service
subset: v1
weight: 90
- destination:
host: product-service
subset: v2
weight: 10
---
apiVersion: networking.istio.io/v1alpha3
kind: DestinationRule
metadata:
name: product-service-dr
spec:
host: product-service
trafficPolicy:
connectionPool:
tcp:
maxConnections: 100
outlierDetection: # Circuit breaking at the mesh level
consecutive5xxErrors: 5
interval: 30s
baseEjectionTime: 30s
subsets:
- name: v1
labels:
version: v1
- name: v2
labels:
version: v2

```

mTLS + AuthorizationPolicy

```

# Enable strict mTLS for the entire namespace
apiVersion: security.istio.io/v1beta1
kind: PeerAuthentication
metadata:
name: default
namespace: microservices
spec:
mtls:

```

```
mode: STRICT # All inter-service traffic must use mTLS

# AuthorizationPolicy - only order-service can call product-service
apiVersion: security.istio.io/v1beta1
kind: AuthorizationPolicy
metadata:
  name: product-service-authz
  namespace: microservices
spec:
  selector:
    matchLabels:
      app: product-service
  rules:
    - from:
        - source:
            principals:
              - "cluster.local/ns/microservices/sa/order-service"
            to:
              - operation:
                  methods: ["GET"]
                  paths: ["/api/products/*"]
```

CHAPTER 3

Kubernetes Deployment for Microservices

Deployments, Services, ConfigMaps, Secrets, HPA & Spring Cloud Kubernetes

3.1 Kubernetes Core Concepts

K8s Object	Purpose	Spring Boot Relevance
Pod	Smallest deployable unit; 1+ containers	One microservice instance
Deployment	Manages ReplicaSets; declarative updates	Defines desired pod count & strategy
Service	Stable DNS name + load balances to pods	Replaces Eureka for K8s-native discovery
ConfigMap	Non-sensitive configuration key-value data	Replaces Spring Cloud Config properties
Secret	Base64-encoded sensitive data (passwords, keys)	DB passwords, JWT secrets
Ingress	HTTP routing rules; external traffic entry point	Replaces API Gateway for K8s
HPA	Auto-scale pods based on CPU/custom metrics	Scale order-service on high load
PersistentVolume	Durable storage that survives pod restarts	Database storage
Namespace	Virtual cluster isolation within a cluster	dev / staging / prod isolation
ServiceAccount	Identity for pods; controls K8s API access	Used by Spring Cloud K8s to read config

3.2 Complete Deployment YAML

Kubernetes Deployment + Service YAML

```
# product-service/k8s/deployment.yml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: product-service
  namespace: microservices
```

```
labels:
app: product-service
version: v1
spec:
replicas: 3
selector:
matchLabels:
app: product-service
strategy:
type: RollingUpdate
rollingUpdate:
maxSurge: 1
maxUnavailable: 0 # Zero-downtime rolling update
template:
metadata:
labels:
app: product-service
version: v1
spec:
serviceAccountName: product-service-sa
containers:
- name: product-service
image: myregistry.io/product-service:1.2.0
ports:
- containerPort: 8082
envFrom:
- configMapRef:
name: product-service-config # non-sensitive config
- secretRef:
name: product-service-secrets # sensitive config
env:
- name: SPRING_PROFILES_ACTIVE
value: "kubernetes"
- name: POD_NAME
valueFrom:
fieldRef:
fieldPath: metadata.name
resources:
requests:
cpu: "250m"
memory: "512Mi"
limits:
cpu: "500m"
memory: "1Gi"
readinessProbe:
```

```

httpGet:
  path: /actuator/health/readiness
  port: 8082
  initialDelaySeconds: 20
  periodSeconds: 10
  failureThreshold: 3
  livenessProbe:
    httpGet:
      path: /actuator/health/liveness
      port: 8082
      initialDelaySeconds: 30
      periodSeconds: 15
  ---
apiVersion: v1
kind: Service
metadata:
  name: product-service
  namespace: microservices
spec:
  selector:
    app: product-service
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8082
  type: ClusterIP # internal only; use Ingress for external access

```

ConfigMap, Secret & HPA

```

# ConfigMap - non-sensitive configuration
apiVersion: v1
kind: ConfigMap
metadata:
  name: product-service-config
  namespace: microservices
data:
  SPRING_DATASOURCE_URL: "jdbc:postgresql://postgres-svc:5432/products"
  SPRING_JPA_SHOW_SQL: "false"
  PRODUCT_CACHE_TTL: "300"
  ---
# Secret - sensitive data (base64 encoded)
apiVersion: v1
kind: Secret
metadata:
  name: product-service-secrets
  namespace: microservices
type: Opaque

```

```

data:
SPRING_DATASOURCE_USERNAME: YWRtaW4= # echo -n "admin" | base64
SPRING_DATASOURCE_PASSWORD: cGFzc3dvcmQ= # echo -n "password" | base64
JWT_SECRET: bXltcXBlc1NlY3JldEtleQ==
---
# HorizontalPodAutoscaler
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
name: product-service-hpa
namespace: microservices
spec:
scaleTargetRef:
apiVersion: apps/v1
kind: Deployment
name: product-service
minReplicas: 2
maxReplicas: 10
metrics:
- type: Resource
resource:
name: cpu
target:
type: Utilization
averageUtilization: 70 # scale up when avg CPU > 70%
- type: Pods
pods:
metric:
name: http_requests_per_second # custom Prometheus metric
target:
type: AverageValue
averageValue: "100"

```

3.3 Spring Cloud Kubernetes

Spring Cloud Kubernetes

Spring Cloud Kubernetes provides Spring Cloud integration with Kubernetes. It replaces Eureka with Kubernetes Service discovery, Spring Cloud Config with ConfigMap-based configuration, and supports automatic configuration reload when ConfigMaps change.

Spring Cloud Kubernetes configuration

```

<!-- pom.xml -->
<dependency>
<groupId>org.springframework.cloud</groupId>
<artifactId>spring-cloud-starter-kubernetes-client-all</artifactId>
</dependency>

```

```
# application-kubernetes.yml
spring:
cloud:
kubernetes:
discovery:
enabled: true # use K8s Service DNS for discovery
all-namespaces: false
config:
enabled: true # load from ConfigMap
sources:
- name: product-service-config
namespace: microservices
reload:
enabled: true # reload config when ConfigMap changes
mode: POLLING
period: 15000 # check every 15 seconds

# RBAC - allow the pod to read ConfigMaps
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
name: product-service-role
namespace: microservices
rules:
- apiGroups: [""]
resources: ["configmaps", "pods", "services", "endpoints"]
verbs: ["get", "list", "watch"]
```


CHAPTER 4

Observability – Prometheus, Grafana & ELK

Metrics, Dashboards, Alerts & Structured Logging

4.1 Micrometer + Prometheus

Micrometer

Micrometer is a metrics facade for JVM applications – the SLF4J of metrics. It collects JVM metrics (heap, GC, threads), HTTP metrics (request count, latency), and custom business metrics. With the Prometheus registry, Spring Boot exposes these at `/actuator/prometheus` in Prometheus text format.

Prometheus setup

```
<!-- pom.xml -->
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>

# application.yml
management:
  endpoints:
    web:
      exposure:
        include: prometheus,health,info,metrics
  metrics:
    tags:
      application: ${spring.application.name} # add app name to all metrics
      environment: ${spring.profiles.active}
  distribution:
    percentiles-histogram:
      http.server.requests: true # enable histogram for latency percentiles
    percentiles:
      http.server.requests: [0.50, 0.95, 0.99]
```

Custom Metrics – Counter, Timer, Gauge

```
// Custom Business Metrics with MeterRegistry
@Service
public class OrderService {

  private final MeterRegistry meterRegistry;
  private final Counter orderCreatedCounter;
```

```

private final Counter orderFailedCounter;
private final Timer orderProcessingTimer;
private final AtomicInteger activeOrders;

public OrderService(MeterRegistry meterRegistry) {
    this.meterRegistry = meterRegistry;
    this.orderCreatedCounter = Counter.builder("orders.created")
        .description("Total orders created")
        .tag("type", "new")
        .register(meterRegistry);
    this.orderFailedCounter = Counter.builder("orders.failed")
        .description("Total orders failed")
        .register(meterRegistry);
    this.orderProcessingTimer = Timer.builder("orders.processing.time")
        .description("Order processing duration")
        .register(meterRegistry);
    this.activeOrders = meterRegistry.gauge("orders.active",
        new AtomicInteger(0));
}

public Order createOrder(OrderRequest req) {
    activeOrders.incrementAndGet();
    return orderProcessingTimer.record(() -> {
        try {
            Order order = processOrder(req);
            orderCreatedCounter.increment();
            return order;
        } catch (Exception e) {
            orderFailedCounter.increment();
            throw e;
        } finally {
            activeOrders.decrementAndGet();
        }
    });
}

// @Timed annotation - auto-creates timer for method
@Service
public class PaymentService {
    @Timed(value = "payment.processing", description = "Payment duration",
        percentiles = {0.5, 0.95, 0.99})
    public PaymentResult processPayment(PaymentRequest req) { ... }
}

```

Prometheus scrape config

```

# prometheus.yml - Scraping Spring Boot services

global:

```

```

scrape_interval: 15s

scrape_configs:
- job_name: "spring-boot-microservices"
metrics_path: "/actuator/prometheus"
static_configs:
- targets:
- "product-service:8082"
- "order-service:8083"
- "payment-service:8084"
# Kubernetes auto-discovery (scrape all pods with annotation)
- job_name: "kubernetes-pods"
kubernetes_sd_configs:
- role: pod
relabel_configs:
- source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_scrape]
action: keep
regex: true
- source_labels: [__meta_kubernetes_pod_annotation_prometheus_io_path]
action: replace
target_label: __metrics_path__

```

4.2 Structured Logging with Logback + ELK

Structured Logging

Structured logging outputs logs as JSON objects instead of plain text strings. Each log entry is a machine-parseable JSON document with standard fields (timestamp, level, service, traceId) and custom fields. This makes logs searchable and aggregatable in Elasticsearch/Kibana without fragile regex parsing.

Logstash encoder dependency

```

<!-- pom.xml -->
<dependency>
<groupId>net.logstash.logback</groupId>
<artifactId>logstash-logback-encoder</artifactId>
<version>7.4</version>
</dependency>

```

Logback JSON configuration

```

<!-- logback-spring.xml - JSON structured logs -->
<configuration>
<springProfile name="kubernetes,prod">
<appender name="JSON" class="ch.qos.logback.core.ConsoleAppender">
<encoder class="net.logstash.logback.encoder.LogstashEncoder">
<customFields>{"service":"${spring.application.name}","env":"${spring.profiles.active}"</cust
omFields>
<fieldNames>
<timestamp>@timestamp</timestamp>
<message>message</message>

```

```
<logger>logger</logger>
<level>level</level>
</fieldNames>
</encoder>
</appender>
<root level="INFO"><appender-ref ref="JSON"/></root>
</springProfile>
</configuration>

// Output (one log event as JSON):
// {
//   "@timestamp": "2025-01-15T10:30:00.000Z",
//   "level": "INFO",
//   "message": "Order created: ORD-123",
//   "service": "order-service",
//   "traceId": "a1b2c3d4e5f6",
//   "spanId": "9f8e7d6c",
//   "orderId": "ORD-123",
//   "customerId": "USR-456"
// }
```

Advanced Kafka Patterns

Kafka Streams, Transactions, DLQ, Schema Registry & Exactly-Once

5.1 Dead Letter Queue (DLQ)

Dead Letter Queue (DLQ)

A DLQ is a special Kafka topic where messages are sent when they cannot be processed successfully after all retries are exhausted. This prevents poison-pill messages from blocking the consumer indefinitely while preserving failed messages for later inspection and reprocessing.

Dead Letter Queue implementation

```
# application.yml - DLQ with Spring Kafka
spring:
  kafka:
    consumer:
      group-id: order-service-group
      listener:
      ack-mode: MANUAL_IMMEDIATE

// DLQ Configuration
@Configuration
public class KafkaConsumerConfig {

  @Bean
  public DefaultErrorHandler errorHandler(KafkaTemplate<String, Object> template) {
    // Retry 3 times with exponential backoff, then send to DLQ
    DeadLetterPublishingRecoverer recoverer =
      new DeadLetterPublishingRecoverer(template,
        (record, ex) -> {
          // DLQ topic = original-topic.DLT
          return new TopicPartition(
            record.topic() + ".DLT",
            record.partition());
        });

    FixedBackOff backOff = new FixedBackOff(1000L, 3);
    ExponentialBackOffWithMaxRetries expBackOff =
      new ExponentialBackOffWithMaxRetries(5);
    expBackOff.setInitialInterval(1000L);
    expBackOff.setMultiplier(2.0);
    expBackOff.setMaxInterval(10000L);

    DefaultErrorHandler handler =
```

```

new DefaultErrorHandler(recoverer, expBackOff);
// Do not retry these exceptions
handler.addNotRetryableExceptions(ValidationException.class);
return handler;
}
}

// DLQ Consumer - process failed messages manually
@Service
public class DlqConsumer {
    @KafkaListener(topics = "order-events.DLT")
    public void handleDeadLetter(
        @Payload OrderPlacedEvent event,
        @Header(KafkaHeaders.EXCEPTION_MESSAGE) String errorMessage,
        @Header(KafkaHeaders.ORIGINAL_TOPIC) String originalTopic) {
        log.error("DLQ message from {}: error={}", originalTopic, errorMessage);
        alertService.notifyDlqMessage(event, errorMessage);
        // Optionally: store in DB for manual reprocessing
    }
}

```

5.2 Kafka Streams – Real-Time Processing

Kafka Streams

Kafka Streams is a client library for building streaming applications where input and output are Kafka topics. It provides stateful processing with windowing, aggregation, joins, and filtering operations. Unlike Spark/Flink, it runs embedded in your Spring Boot application with no separate cluster needed.

Kafka Streams – Revenue Aggregation with Windowing

```

<!-- pom.xml -->
<dependency>
    <groupId>org.springframework.kafka</groupId>
    <artifactId>spring-kafka</artifactId>
</dependency>

// Real-time order revenue aggregation using Kafka Streams
@Configuration
@EnableKafkaStreams
public class OrderStreamConfig {

    @Bean(name = KafkaStreamsDefaultConfiguration.DEFAULT_STREAMS_CONFIG_BEAN_NAME)
    public KafkaStreamsConfiguration streamsConfig() {
        Map<String, Object> props = new HashMap<>();
        props.put(StreamsConfig.APPLICATION_ID_CONFIG, "order-analytics");
        props.put(StreamsConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
        props.put(StreamsConfig.DEFAULT_KEY_SERDE_CLASS_CONFIG, Serdes.String().getClass());
        props.put(StreamsConfig.DEFAULT_VALUE_SERDE_CLASS_CONFIG, Serdes.Double().getClass());
        return new KafkaStreamsConfiguration(props);
    }
}

```

```
@Bean
public KStream<String, OrderEvent> orderRevenueStream(StreamsBuilder builder) {
    JsonSerde<OrderEvent> orderSerde = new JsonSerde<>(OrderEvent.class);

    KStream<String, OrderEvent> orders =
        builder.stream("order-events", Consumed.with(Serdes.String(), orderSerde));

    // Aggregate total revenue per category per 1-minute tumbling window
    orders
        .filter((key, order) -> "CONFIRMED".equals(order.getStatus()))
        .map((key, order) ->
            KeyValue.pair(order.getCategory(), order.getTotalAmount()))
        .groupByKey(Grouped.with(Serdes.String(), Serdes.Double()))
        .windowedBy(TimeWindows.ofSizeWithNoGrace(Duration.ofMinutes(1)))
        .reduce(Double::sum)
        .toStream()
        .map((windowedKey, revenue) -> KeyValue.pair(
            windowedKey.key(),
            new RevenueEvent(windowedKey.key(), revenue,
                windowedKey.window().startTime())))
        .to("order-revenue-by-category",
            Produced.with(Serdes.String(), new JsonSerde<>(RevenueEvent.class)));

    return orders;
}
}
```

CHAPTER 6

Transactional Outbox Pattern

Solving the Dual-Write Problem – Guaranteed Event Publishing with Debezium

6.1 The Dual-Write Problem

Dual-Write Problem

In microservices, a service often needs to update its database AND publish an event to Kafka in the same operation. If the DB commit succeeds but the Kafka publish fails (or vice versa), your data and events are inconsistent. Two-phase commit (2PC) across a DB and Kafka is impractical and not supported.

x DANGER: This code has the dual-write bug – event may be lost if Kafka is down after the DB commit, or published before the DB commit is visible.

Anti-Pattern: Dual-Write Bug

```
// WRONG: Dual-write bug
@Transactional
public Order createOrder(OrderRequest req) {
    Order order = orderRepo.save(new Order(req)); // 1. Save to DB
    kafkaTemplate.send("order-events", event); // 2. Publish to Kafka
    // If Kafka is down at step 2: order saved, event LOST!
    // If DB rolls back after step 2: event published, order MISSING!
    return order;
}
```

6.2 Outbox Pattern Architecture

- **Step 1** – In the same DB transaction, save the business entity AND insert a row into an **outbox table**.
- **Step 2** – A separate process (CDC or polling) reads new rows from the outbox table.
- **Step 3** – The reader publishes events to Kafka and marks the outbox row as processed.
- **Atomicity** – DB transaction guarantees both the entity save and outbox insert happen together.
- **At-Least-Once** – If publish fails, the outbox row is retried until Kafka acknowledges.

Outbox Pattern – Transactional Save + Polling Publisher

```
// Outbox entity
@Entity
@Table(name = "outbox_events")
public class OutboxEvent {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String aggregateType; // e.g. "Order"
    private String aggregateId; // e.g. order UUID
}
```



```

private String eventType; // e.g. "OrderPlacedEvent"
private String topic; // Kafka topic
@Column(columnDefinition = "TEXT")
private String payload; // JSON serialised event
private LocalDateTime createdAt = LocalDateTime.now();
private boolean processed = false;
}

// Save order + outbox row in ONE transaction
@Service
@Transactional
public class OrderService {
    private final OrderRepository orderRepo;
    private final OutboxRepository outboxRepo;
    private final ObjectMapper objectMapper;

    public Order createOrder(OrderRequest req) {
        // 1. Save business entity
        Order order = orderRepo.save(new Order(req));

        // 2. Save outbox event in SAME transaction
        OrderPlacedEvent event = new OrderPlacedEvent(
            order.getId().toString(), order.getItems(), order.getTotalAmount());
        outboxRepo.save(OutboxEvent.builder()
            .aggregateType("Order")
            .aggregateId(order.getId().toString())
            .eventType("OrderPlacedEvent")
            .topic("order-events")
            .payload(objectMapper.writeValueAsString(event))
            .build());

        // NO Kafka call here - Debezium/scheduler handles publishing
        return order;
    }
}

// Polling publisher (alternative to CDC)
@Service
public class OutboxPublisher {
    @Scheduled(fixedDelay = 1000) // every second
    @Transactional
    public void publishPendingEvents() {
        List<OutboxEvent> pending = outboxRepo.findByProcessedFalseOrderByCreatedAtAsc();
        for (OutboxEvent event : pending) {
            kafkaTemplate.send(event.getTopic(), event.getAggregateId(), event.getPayload())
                .whenComplete((r, ex) -> {
                    if (ex == null) {
                        event.setProcessed(true);
                        outboxRepo.save(event);
                    }
                });
        }
    }
}

```

```
}  
});  
}  
}  
}
```

i For production, use Debezium CDC (Change Data Capture) instead of polling. Debezium reads the database WAL (Write-Ahead Log) and publishes outbox rows to Kafka automatically with zero polling latency and exactly-once delivery.

Event Sourcing & Axon Framework

7.1 Event Sourcing Concepts

Event Sourcing is an architectural pattern where the state of an entity is stored as a sequence of immutable domain events, rather than its current state. To get the current state, you replay all events. This gives you a complete audit trail, the ability to rebuild state at any point in time, and decoupled read models via projections.

7.2 Axon Framework – Aggregate, Command & Event

Axon Framework is a Java framework for building event-driven, event-sourced, and CQRS applications. It provides Aggregates (domain objects), Command Handlers, Event Handlers, Sagas, an Event Store, and integrates with Spring Boot. Axon Server can act as both Event Store and Message Broker.

```
<!-- pom.xml -->
<dependency>
<groupId>org.axonframework</groupId>
<artifactId>axon-spring-boot-starter</artifactId>
<version>4.9.0</version>
</dependency>
```

```
// █ Commands ██████████
// Commands express INTENT - what the user wants to do

public record CreateOrderCommand(
    @TargetAggregateIdentifier String orderId,
```

[illegible]


```
}  
}  
  
// Command Gateway - send commands from controllers  
@RestController  
@RequestMapping("/api/orders")  
public class OrderController {  
    private final CommandGateway commandGateway;  
    private final QueryGateway queryGateway;  
  
    @PostMapping  
    public CompletableFuture<String> create(@RequestBody CreateOrderRequest req) {  
        String orderId = UUID.randomUUID().toString();  
        return commandGateway.send(new CreateOrderCommand(  
            orderId, req.customerId(), req.items(), req.totalAmount()));  
    }  
  
    @GetMapping("/{id}")  
    public CompletableFuture<OrderView> get(@PathVariable String id) {  
        return queryGateway.query(new FindOrderQuery(id), OrderView.class);  
    }  
}
```

CHAPTER 8

GraphQL API Gateway

Federated GraphQL Schema, DataLoader & Subscriptions with Spring GraphQL

8.1 Why GraphQL at the Gateway?

GraphQL

GraphQL is a query language for APIs where clients specify exactly what data they need. At the API Gateway level, GraphQL can aggregate data from multiple microservices in a single request, eliminating the N+1 problem of multiple REST calls from the frontend, and providing a single, typed schema for all client applications.

Aspect	REST API Gateway	GraphQL Gateway
Data Fetching	Multiple REST calls for related data	Single query, server aggregates
Over-fetching	Fixed response shape; returns all fields	Client specifies exact fields needed
Under-fetching	Multiple round-trips for related data	Nested queries in one request
Type Safety	Optional (OpenAPI)	Built-in schema & type system
Versioning	URL-based (/v1, /v2)	Evolve schema without versioning

Spring GraphQL dependency

```
<!-- pom.xml -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-graphql</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-webflux</artifactId>
</dependency>
```

GraphQL Schema Definition

```
# src/main/resources/graphql/schema.graphqls
type Query {
  product(id: ID!): Product
  products(category: String, page: Int, size: Int): ProductPage!
  order(id: ID!): Order
  customerOrders(customerId: ID!): [Order!]!
}

type Mutation {
```

```

createOrder(input: CreateOrderInput!): Order!
cancelOrder(orderId: ID!, reason: String): Order!
}

type Subscription {
  orderStatusUpdated(orderId: ID!): Order!
}

type Product {
  id: ID!
  name: String!
  price: Float!
  stock: Int!
  category: String!
}

type Order {
  id: ID!
  status: String!
  totalAmount: Float!
  items: [OrderItem!]!
  customer: Customer! # Fetched from user-service
  product: Product # Fetched from product-service
}

type Customer {
  id: ID!
  name: String!
  email: String!
}

input CreateOrderInput {
  customerId: ID!
  items: [OrderItemInput!]!
}

```

GraphQL Controller, Nested Resolver & DataLoader

```

// GraphQL Resolver (Controller)
@Controller
public class OrderGraphQLController {

  private final OrderServiceClient orderClient;
  private final UserServiceClient userClient;

  @QueryMapping
  public Order order(@Argument String id) {
    return orderClient.getOrder(id);
  }

  @MutationMapping
  public Order createOrder(@Argument CreateOrderInput input) {
    return orderClient.createOrder(input);
  }
}

```



```

}

// Resolve nested Customer field on Order
// Called for EVERY Order in a list - DataLoader prevents N+1
@SchemaMapping(typeName = "Order", field = "customer")
public CompletableFuture<Customer> customer(Order order,
DataLoader<String, Customer> loader) {
// DataLoader batches all customer IDs and fetches in one call
return loader.load(order.getCustomerId());
}

// Subscription - real-time order status updates
@SubscriptionMapping
public Flux<Order> orderStatusUpdated(@Argument String orderId) {
return orderClient.subscribeToOrderUpdates(orderId);
}
}

// DataLoader - batches N customer lookups into 1 call
@Component
public class CustomerDataLoader implements BatchLoaderWithContext<String, Customer> {
private final UserServiceClient userClient;

@Override
public CompletionStage<List<Customer>> load(List<String> ids,
BatchLoaderEnvironment env) {
// One bulk call for ALL customer IDs in this batch
return CompletableFuture.supplyAsync()
-> userClient.getUsersByIds(ids));
}
}

```

CHAPTER 9

Multi-Tenancy in Microservices

Tenant Isolation Strategies, Context Propagation & Row-Level Security

9.1 Multi-Tenancy Strategies

Strategy	Isolation Level	Pros	Cons
Database per Tenant	Highest	Full isolation, easy compliance	Expensive, schema management burden
Schema per Tenant	Medium	Good isolation, shared DB host	Schema migration for all tenants
Row-Level Security	Lowest	Cost-effective, simple ops	Risk of data leaks if query bugs
Hybrid	Flexible	Premium tenants get own DB	Complex routing logic

9.2 Tenant Context Propagation

Multi-Tenancy: TenantContext + Filter + DataSource Routing + Feign Propagation

```
// 1. TenantContext - ThreadLocal storage
public class TenantContext {
    private static final ThreadLocal<String> CURRENT_TENANT =
        new InheritableThreadLocal<>();

    public static void setCurrentTenant(String tenantId) {
        CURRENT_TENANT.set(tenantId);
    }

    public static String getCurrentTenant() {
        return CURRENT_TENANT.get();
    }

    public static void clear() {
        CURRENT_TENANT.remove();
    }
}

// 2. Tenant Filter - extracts tenant from JWT or header
@Component
@Order(Ordered.HIGHEST_PRECEDENCE)
public class TenantFilter extends OncePerRequestFilter {
    @Override
    protected void doFilterInternal(HttpServletRequest request,
        HttpServletResponse response, FilterChain chain)
        throws ServletException, IOException {
```

```

// Extract from header X-Tenant-Id or from JWT claim
String tenantId = request.getHeader("X-Tenant-Id");
if (tenantId == null) {
    // Extract from JWT "tenantId" claim
    tenantId = extractTenantFromJwt(request);
}
if (tenantId == null) {
    response.sendError(400, "Missing tenant identifier");
    return;
}
TenantContext.setCurrentTenant(tenantId);
try {
    chain.doFilter(request, response);
} finally {
    TenantContext.clear(); // CRITICAL: clear after request
}
}
}

// 3. Dynamic DataSource Routing per tenant
@Component
public class TenantRoutingDataSource extends AbstractRoutingDataSource {
    @Override
    protected Object determineCurrentLookupKey() {
        return TenantContext.getCurrentTenant();
    }
}

// 4. Feign interceptor - propagate X-Tenant-Id to downstream services
@Component
public class TenantFeignInterceptor implements RequestInterceptor {
    @Override
    public void apply(RequestTemplate template) {
        String tenantId = TenantContext.getCurrentTenant();
        if (tenantId != null) {
            template.header("X-Tenant-Id", tenantId);
        }
    }
}

```

Advanced Deployment Patterns

Blue-Green, Canary Releases, Feature Flags & Zero-Downtime DB Migrations

10.1 Blue-Green Deployment

Blue-Green Deployment

Blue-Green deployment maintains two identical production environments (Blue = current, Green = new version). Traffic is switched instantly from Blue to Green via load balancer update. If the new version has issues, you roll back by switching back to Blue in seconds.

Blue-Green Deployment with Kubernetes

```
# Kubernetes - Blue-Green with Service selector switch

# Blue deployment (current)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-service-blue
spec:
  replicas: 3
  selector:
    matchLabels:
      app: order-service
      version: blue
  template:
    metadata:
      labels:
        app: order-service
        version: blue
    spec:
      containers:
        - name: order-service
          image: order-service:1.0.0
  ---

# Green deployment (new version - deployed but no traffic yet)
apiVersion: apps/v1
kind: Deployment
metadata:
  name: order-service-green
spec:
  replicas: 3
```

```

selector:
matchLabels:
  app: order-service
  version: green
template:
metadata:
  labels:
    app: order-service
    version: green
spec:
  containers:
    - name: order-service
      image: order-service:2.0.0
  ---
# Service - switch traffic by updating selector
apiVersion: v1
kind: Service
metadata:
  name: order-service
spec:
  selector:
    app: order-service
    version: blue # change to "green" to switch traffic

# Switch: kubectl patch svc order-service -p
# '{"spec":{"selector":{"version":"green"}}}'
# Rollback: change version back to "blue"

```

10.2 Feature Flags with Spring

Feature Flag (Feature Toggle)

A feature flag is a configuration value that enables or disables a code feature at runtime without redeploying. This allows dark launches, A/B testing, and gradual rollouts. Spring Boot integrates with FF4J, Unleash, or LaunchDarkly for dynamic flags.

Feature Flags with FF4J

```

<!-- FF4J - Feature Flags for Java -->
<dependency>
<groupId>org.ff4j</groupId>
<artifactId>ff4j-spring-boot-starter</artifactId>
<version>2.1.0</version>
</dependency>

// FF4J Configuration
@Configuration
public class FeatureFlagConfig {
    @Bean
    public FF4j ff4j() {

```

```

return new FF4j()
.createFeature("new-payment-flow")
.createFeature("ai-recommendations", false)
.createFeature("express-checkout", true);
}
}

// Usage in Service
@Service
public class PaymentService {
private final FF4j ff4j;

public PaymentResult processPayment(PaymentRequest req) {
if (ff4j.check("new-payment-flow")) {
return newPaymentFlow.process(req); // new feature
} else {
return legacyPaymentFlow.process(req); // old feature
}
}
}
}

```

10.3 Zero-Downtime Database Migrations with Flyway

Zero-Downtime Flyway Migrations – Expand-Contract Pattern

```

<!-- pom.xml -->
<dependency>
<groupId>org.flywaydb</groupId>
<artifactId>flyway-core</artifactId>
</dependency>

# Flyway migration files - src/main/resources/db/migration/
# Naming: V{version}__{description}.sql

-- V1__create_orders_table.sql
CREATE TABLE orders (
id BIGSERIAL PRIMARY KEY,
customer_id VARCHAR(36) NOT NULL,
status VARCHAR(20) NOT NULL DEFAULT 'PENDING',
total_amount DECIMAL(10,2) NOT NULL,
created_at TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- V2__add_order_reference.sql (backward compatible: nullable first)
ALTER TABLE orders ADD COLUMN reference_number VARCHAR(20);

-- V3__populate_reference_number.sql
UPDATE orders SET reference_number = 'ORD-' || LPAD(id::TEXT, 8, '0')
WHERE reference_number IS NULL;

-- V4__make_reference_not_null.sql (only after V3 backfill is safe)
ALTER TABLE orders ALTER COLUMN reference_number SET NOT NULL;

```

```
CREATE UNIQUE INDEX idx_orders_reference ON orders (reference_number);

# application.yml - Flyway auto-runs on startup
spring:
  flyway:
    enabled: true
    locations: classpath:db/migration
    baseline-on-migrate: true
```

Master Quick Reference – Advanced Spring Cloud

Technology / Pattern	Purpose	Key Dependency / Tool
gRPC	High-perf binary inter-service RPC	grpc-server/client-spring-boot-starter
Protocol Buffers	gRPC serialisation format / schema	.proto files + protobuf-maven-plugin
Istio VirtualService	Traffic splitting, retries, timeouts	Kubernetes + Istio CRDs
Istio mTLS	Automatic mutual TLS between services	PeerAuthentication YAML
Spring Cloud K8s	Kubernetes-native service discovery + config	spring-cloud-starter-kubernetes-client-all
Kubernetes HPA	Auto-scale pods on CPU/custom metrics	autoscaling/v2 YAML
Micrometer Prometheus	Metrics export to Prometheus	micrometer-registry-prometheus
Structured Logging	JSON logs for ELK/Splunk ingestion	logstash-logback-encoder
Kafka DLQ	Handle unprocessable messages	DeadLetterPublishingRecoverer
Kafka Streams	Real-time stateful stream processing	spring-kafka + @EnableKafkaStreams
Outbox Pattern	Guaranteed event publishing (no dual-write)	Outbox table + Debezium CDC
Axon Framework	Event sourcing, CQRS, distributed sagas	axon-spring-boot-starter
Spring GraphQL	Federated GraphQL API with DataLoader	spring-boot-starter-graphql
Multi-Tenancy	Tenant isolation with dynamic DataSource routing	AbstractRoutingDataSource
Blue-Green Deploy	Zero-risk deployments with instant rollback	Kubernetes Service selector patch
Feature Flags	Runtime feature toggling without redeploy	FF4J / Unleash / LaunchDarkly
Flyway	Version-controlled zero-downtime DB migrations	flyway-core

+ Volume 1 covered: Eureka, API Gateway, Config Server, Resilience4j, Kafka basics, RabbitMQ, Zipkin tracing, Docker Compose, and Saga/CQRS patterns. This Volume 2 builds on those fundamentals with production-grade advanced patterns.

! gRPC is excellent for internal service communication but requires careful versioning – adding fields to .proto is safe (forward compatible), but removing or renumbering fields breaks existing clients. Always evolve schemas additively.